

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

Faculty Name: Dr.G.Ayyappan

Student Reg. No: 192571049

Name: Hemalakshmi

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Sequential File vs Heap File (Analyze)

Sequential File:

- Records stored in sorted order → retrieval is efficient using binary search ($O(\log N)$).
- Insertions and deletions require shifting records to maintain order → $O(N)$ cost.
- Excellent for range queries and ordered reports.

Heap File:

- Records stored in insertion order → insertion is $O(1)$.
- Deletion and retrieval require linear search → $O(N)$.
- Suitable for high-volume inserts but poor for search.

Analysis: Sequential files optimize retrieval; heap files optimize insertion.

2. Clustered Index vs Non-Clustered Index (Analyze)

Clustered Index:

- Physically orders the data based on the index key.
- Only one allowed per table.
- Faster retrieval due to locality of data pages.
- Example: Table sorted by `order_id`.

Non-Clustered Index:

- Stores pointers to data; physical order unchanged.
- Multiple allowed.
- Retrieval requires extra I/O to follow pointers.
- Example: Index on `customer_name`.

Analysis: Clustered index improves sequential access; non-clustered index improves flexible attribute-based search.

3. Primary Index vs Secondary Index (Analyze)

Primary Index:

- Built on primary key; file sorted on that key.
- Sparse index → fewer entries → low storage.
- Fast search using binary search.
- Supports range queries efficiently.

Secondary Index:

- Built on non-key attributes; file not sorted on that attribute.
- Dense index → more entries → higher storage.
- Search requires index lookup + pointer follow.
- Supports queries on non-key fields.

Analysis: Primary index is storage-efficient; secondary index is flexible but costlier.

4. Static Hashing vs Dynamic Hashing (Analyze)

Static Hashing:

- Fixed number of buckets.
- Overflow chains grow when distribution is skewed.
- Good for stable datasets.
- Poor scalability.

Dynamic Hashing:

- Buckets split dynamically based on load.

- Directory grows/shrinks → avoids long overflow chains.
- Supports large, evolving datasets.
- Better performance under skew.

Analysis: Static hashing is simple; dynamic hashing is scalable and adaptive.

5. B-Tree vs B+ Tree Indexing (Analyze)

B-Tree:

- Keys stored in internal and leaf nodes.
- Search may end at internal node.
- Range queries slower due to no leaf linking.

B+ Tree:

- All keys stored only in leaves; internal nodes store routing keys.
- Leaves linked → fast range scans.
- Higher fan-out → shorter height → fewer I/Os.
- Preferred in large DBMS.

Analysis: B+ Tree is superior for large datasets and range queries.

6. Linear Hashing vs Extendible Hashing (Analyze)

Linear Hashing:

- Buckets split in a predictable linear sequence.
- No directory; simpler structure.
- Gradual growth → avoids sudden doubling.
- May cause temporary imbalance.

Extendible Hashing:

- Uses directory that doubles when needed.
- Splits only the overflowing bucket.
- More balanced distribution.
- Directory maintenance adds overhead.

Analysis: Linear hashing grows smoothly; extendible hashing maintains better bucket balance.

7. Dense Index vs Sparse Index (Analyze)

Dense Index:

- Entry for every record.
- High memory usage.
- Fast access because index directly maps to record.
- Suitable for unsorted files.

Sparse Index:

- Entry for every block.
- Low memory usage.
- Requires block scan after locating block.
- Suitable for sorted files.

Analysis: Dense index is faster; sparse index is more space-efficient.

8. Indexed Sequential File vs Direct File Organization (Analyze)

Indexed Sequential File:

- File sorted; index supports fast lookup.
- Excellent for range queries and ordered reports.

- Insert/delete expensive due to maintaining order.
- More storage due to index overhead.

Direct File (Hashing):

- Uses hash function for direct access.
- Fast exact-match queries.
- Poor for range queries.
- Overflow buckets reduce performance.

Analysis: Indexed sequential is best for ranges; direct file is best for exact matches.

9. Single-Level Indexing vs Multi-Level Indexing (Analyze)

Single-Level Index

- One index structure.
- Search cost $O(N)$ for large index files.
- Suitable for small datasets.

Multi-Level Index:

- Index on index $\rightarrow$ hierarchical structure.
- Search cost reduces to $O(\log N)$.
- Supports large datasets efficiently.
- Fewer disk accesses.

Analysis: Multi-level indexing dramatically improves search performance for large files.

10. Hashing vs Indexing for Exact-Match & Range Queries (Evaluate)

Hashing:

- Exact-match queries $\rightarrow O(1)$ average.
- Range queries $\rightarrow$ inefficient; no ordering.
- Best for equality searches (ID, account number).

Indexing (B+ Tree):

- Exact-match $\rightarrow O(\log N)$.
- Range queries $\rightarrow$ very efficient due to leaf linking.
- Supports ordered scans and prefix searches.

Evaluation:

- Hashing is best for exact matches.
- Indexing is best for range queries and ordered retrieval.
- For mixed workloads, B+ Tree indexing provides more balanced performance.